

LOVE this. 🔥

This is exactly how you become strong at grammar design.

We're going to build this **like a real system designer** — step by step — and then combine everything into one FULL BNF grammar at the end.

Remember:

BNF describes **structure**, not logic execution.
It defines *possible valid system states and decisions*.

✂ STEP 1 — DEFINE INPUTS (Monitored Factors)

We start by defining each monitored factor as a non-terminal.

◆ Altitude

```
<altitude> ::= "low" | "normal" | "high"
```

◆ Pitch

```
<pitch> ::= "up" | "level" | "down"
```

◆ Error Margin

```
<error> ::= "low" | "moderate" | "high"
```

◆ Weather

```
<weather> ::= "clear" | "storm" | "turbulence"
```

So now we have all inputs defined.

✂ STEP 2 — DEFINE SYSTEM STATE INPUT

We need a structure that represents the full monitored state.

For example:

```
Altitude: low
Pitch: up
Error: high
Weather: storm
```

So we define a monitored state rule:

```
<state> ::= "Altitude=" <altitude> ", "
          "Pitch=" <pitch> ", "
          "Error=" <error> ", "
          "Weather=" <weather>
```

Now we can describe a full system condition.

Example valid sentence:

```
Altitude=low, Pitch=up, Error=high, Weather=storm
```

✂ STEP 3 — DEFINE SYSTEM DECISION

We define possible system actions.

The system can:

- Auto-correct
- Trigger alert
- Allow pilot override

So:

```
<decision> ::= <auto_correct>
              | <alert>
              | <override>
```

◆ Auto-correct rule

Auto-correct happens if error is low.

BNF cannot "calculate" logic — so we define valid pattern:

```
<auto_correct> ::= <low_error_state> "->" "AUTO_CORRECT"
```

Where:

```
<low_error_state> ::= "Altitude=" <altitude> ", "  
                    "Pitch=" <pitch> ", "  
                    "Error=low, "  
                    "Weather=" <weather>
```

◆ Alert rule (when error high)

```
<alert> ::= <high_error_state> "->" <alert_type>
```

Where:

```
<high_error_state> ::= "Altitude=" <altitude> ", "  
                    "Pitch=" <pitch> ", "  
                    "Error=high, "  
                    "Weather=" <weather>
```

◆ Manual Override

```
<override> ::= "PILOT_OVERRIDE"
```

STEP 4 — DEFINE SPECIFIC ALERT TYPES

Now we define alert categories.

Low Fuel Warning

Altitude = low & Error = high

```
<low_fuel_warning> ::=  
    "Altitude=low, Pitch=" <pitch> ", Error=high, Weather=" <weather>  
    "-> LOW_FUEL_WARNING"
```

Stall Warning

Pitch = up & Error = moderate or high

```
<stall_warning> ::=  
    "Altitude=" <altitude> ", Pitch=up, Error=moderate, Weather=" <weather>  
    "-> STALL_WARNING"  
|  
    "Altitude=" <altitude> ", Pitch=up, Error=high, Weather=" <weather>  
    "-> STALL_WARNING"
```

Severe Turbulence Alert

Weather = turbulence & Error = high

```
<severe_turbulence> ::=  
    "Altitude=" <altitude> ", Pitch=" <pitch> ", Error=high,  
    Weather=turbulence"  
    "-> SEVERE_TURBULENCE_ALERT"
```

Critical Weather Alert

Weather = storm

```
<critical_weather> ::=  
    "Altitude=" <altitude> ", Pitch=" <pitch> ", Error=" <error> ",  
    Weather=storm"  
    "-> CRITICAL_WEATHER_ALERT"
```

FINAL FULL BNF GRAMMAR

Now we combine EVERYTHING into one clean grammar.

```
<system> ::= <decision>  
  
<decision> ::= <auto_correct>  
              | <alert_case>  
              | <override>
```

```

<auto_correct> ::=
    "Altitude=" <altitude> ", "
    "Pitch=" <pitch> ", "
    "Error=low, "
    "Weather=" <weather>
    "-> AUTO_CORRECT"

<alert_case> ::= <low_fuel_warning>
                | <stall_warning>
                | <severe_turbulence>
                | <critical_weather>

<override> ::= "PILOT_OVERRIDE"

<low_fuel_warning> ::=
    "Altitude=low, Pitch=" <pitch> ", Error=high, Weather=" <weather>
    "-> LOW_FUEL_WARNING"

<stall_warning> ::=
    "Altitude=" <altitude> ", Pitch=up, Error=moderate, Weather=" <weather>
    "-> STALL_WARNING"
    |
    "Altitude=" <altitude> ", Pitch=up, Error=high, Weather=" <weather>
    "-> STALL_WARNING"

<severe_turbulence> ::=
    "Altitude=" <altitude> ", Pitch=" <pitch> ", Error=high,
Weather=turbulence"
    "-> SEVERE_TURBULENCE_ALERT"

<critical_weather> ::=
    "Altitude=" <altitude> ", Pitch=" <pitch> ", Error=" <error> ",
Weather=storm"
    "-> CRITICAL_WEATHER_ALERT"

<altitude> ::= "low" | "normal" | "high"

<pitch> ::= "up" | "level" | "down"

<error> ::= "low" | "moderate" | "high"

<weather> ::= "clear" | "storm" | "turbulence"

```

```

<system> ::= <monitoring> <decision>

```

```

<monitoring> ::=
    "Altitude=" <altitude>

```

"Pitch=" <pitch>

"Error=" <error_margin>

"Weather=" <weather>

<altitude> ::= "low" | "normal" | "high"

<pitch> ::= "up" | "level" | "down"

<error_margin> ::= "low" | "moderate" | "high"

<weather> ::= "clear" | "storm" | "turbulence"

<decision> ::= <autocorrect>

| <alert>

| <pilot_override>

<autocorrect> ::=

"AUTOCORRECT_INITIATED"

"WHEN"

"Error=low"

<alert> ::= <low_fuel_warning>

| <stall_warning>

| <severe_turbulence_alert>

| <critical_weather_alert>

```
<low_fuel_warning> ::=  
  "LOW_FUEL_WARNING"  
  "WHEN"  
  "Altitude=low AND Error=high"
```

```
<stall_warning> ::=  
  "STALL_WARNING"  
  "WHEN"  
  "Pitch=up AND Error=moderate"  
  |  
  "STALL_WARNING"  
  "WHEN"  
  "Pitch=up AND Error=high"
```

```
<severe_turbulence_alert> ::=  
  "SEVERE_TURBULENCE_ALERT"  
  "WHEN"  
  "Weather=turbulence AND Error=high"
```

```
<critical_weather_alert> ::=  
  "CRITICAL_WEATHER_ALERT"  
  "WHEN"  
  "Weather=storm"
```

<pilot_override> ::=

"MANUAL_OVERRIDE"

"WHEN"

"pilot_takes_control"

<system> ::= <monitoring> <decision>

<monitoring> ::=

"HeartRate=" <heart_rate>

"BloodPressure=" <blood_pressure>

"Temperature=" <temperature>

"Consciousness=" <consciousness>

<heart_rate> ::= "low" | "normal" | "high"

<blood_pressure> ::= "low" | "normal" | "high"

<temperature> ::= "normal" | "fever" | "critical"

<consciousness> ::= "alert" | "drowsy" | "unconscious"

<decision> ::= <auto_stabilize>

| <alert_case>

| <manual_override>

<auto_stabilize> ::= "AUTO_STABILIZE_INITIATED WHEN " <stabilize_condition>

<stabilize_condition> ::= "HeartRate=low" | "BloodPressure=low"

<alert_case> ::= <cardiac_alert>

| <critical_fever_alert>

| <coma_alert>

<cardiac_alert> ::= "CARDIAC_ALERT WHEN HeartRate=high AND BloodPressure=high"

<critical_fever_alert> ::= "CRITICAL_FEVER_ALERT WHEN Temperature=critical"

<coma_alert> ::= "COMA_ALERT WHEN Consciousness=unconscious"

<manual_override> ::= "MANUAL_OVERRIDE WHEN doctor_takes_control"

Scenario-Based Activity

Scenario: Smart Home Automation Language

Imagine you are designing a DSL for a Smart Home Automation System. This DSL will allow users to control household devices using a simple set of commands. Design the syntax of this DSL with BNF grammar.

Supported Devices & Features:

Devices: Lights, Thermostat, Security System, Smart Locks

Input Types: Motion Sensors, Time-based Triggers, Manual Commands

Features:

Turn devices ON/OFF

Set temperature for thermostat

Lock/unlock doors

Automate actions based on sensor input

Example Commands

Basic command: TURN ON LIVING ROOM LIGHTS

Parameterized command: SET TEMPERATURE BEDROOM TO 22

Conditional command: IF MOTION DETECTED THEN TURN ON LIGHTS

Sequential command: TURN OFF KITCHEN LIGHTS THEN LOCK FRONT DOOR

Loop-based automation: WHILE TIME < 22:00 KEEP LIGHTS ON

<command> ::= <basic_command>

| <parameterized_command>

| <conditional_command>

| <sequential_command>

| <loop_command>

<basic_command> ::= "TURN_ON" <location> <device>

| "TURN_OFF" <location> <device>

<parameterized_command> ::= "SET_TEMPERATURE" <location> "TO" <temperature_value>

| "LOCK" <location>

| "UNLOCK" <location>

<conditional_command> ::= "IF" <trigger> "THEN" <command>

<sequential_command> ::= <command> "THEN" <command>

<loop_command> ::= "WHILE" <condition> "KEEP" <command>

<condition> ::= "TIME" "<" <time>

| "TIME" ">" <time>

<device> ::= "LIGHTS" | "THERMOSTAT" | "SECURITY_SYSTEM" | "SMART_LOCKS"

<location> ::= "LIVING_ROOM" | "BEDROOM" | "KITCHEN" | "FRONT_DOOR"

<trigger> ::= "MOTION_DETECTED" | "TIME" | "MANUAL_COMMAND"

<temperature_value> ::= <multi_digit_number>

<time> ::= <multi_digit_number> ":" <multi_digit_number>

<number> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

<multi_digit_number> ::= <number> | <number> <multi_digit_number>

SCENARIO 3 — Smart Factory Automation DSL

Imagine you are designing a **Domain-Specific Language (DSL)** for a **Smart Factory**. The DSL will allow operators to control machines, monitor sensors, and schedule tasks.

1 Supported Devices & Machines

- Conveyor Belt
 - Robotic Arm
 - Furnace
 - Cooling System
-

2 Input Types / Triggers

- Sensor readings: TEMPERATURE, PRESSURE, VIBRATION

- Time-based events: SCHEDULED_TIME
 - Manual commands: MANUAL_OVERRIDE
-

3 Actions / Features

- Start / Stop machines
 - Set temperature for Furnace
 - Move Robotic Arm to a position
 - Activate / Deactivate Cooling System
 - Conditional operations based on sensor readings
 - Sequential commands (execute multiple commands one after another)
 - Looping commands until a condition is met
-

4 Example Commands

1. **Basic command:** START CONVEYOR_BELT
 2. **Parameterized command:** SET FURNACE TEMPERATURE TO 200
 3. **Conditional command:** IF TEMPERATURE > 150 THEN ACTIVATE COOLING_SYSTEM
 4. **Sequential command:** STOP ROBOTIC_ARM THEN START CONVEYOR_BELT
 5. **Loop-based automation:** WHILE VIBRATION < 5 KEEP FURNACE ON
-

? Task

Write a **BNF grammar** to describe this DSL.

- Define **machines, triggers, actions, conditions**
- Handle **conditional, sequential, and loop-based commands**
- Use modular design with **reusable rules**
- Use proper **BNF syntax** (::=)

```
<command> ::= <basic_command>
           | <parameterized_command>
           | <conditional_command>
           | <sequential_command>
           | <loop_command>
```

<basic_command> ::= "START" <machine>

| "STOP" <machine>

| "ACTIVATE" <machine>

| "DEACTIVATE" <machine>

<parameterized_command> ::= "SET" "FURNACE" "TEMPERATURE" "TO" <temperature_value>

| "MOVE_ARM" "TO" <position_value>

<conditional_command> ::= "IF" <condition> "THEN" <command>

<sequential_command> ::= <command> "THEN" <command>

<loop_command> ::= "WHILE" <condition> "KEEP" <command>

<condition> ::= <trigger> ">" <sensor_threshold>

| <trigger> "<" <sensor_threshold>

<machine> ::= "CONVEYOR_BELT" | "ROBOTIC_ARM" | "FURNACE" | "COOLING_SYSTEM"

<trigger> ::= "TEMPERATURE" | "PRESSURE" | "VIBRATION" | "SCHEDULED_TIME" |
"MANUAL_OVERRIDE"

<temperature_value> ::= <multi_digit_number>

<position_value> ::= <multi_digit_number>

<sensor_threshold> ::= <multi_digit_number>

$\langle \text{number} \rangle ::= "0" \mid "1" \mid "2" \mid "3" \mid "4" \mid "5" \mid "6" \mid "7" \mid "8" \mid "9"$

$\langle \text{multi_digit_number} \rangle ::= \langle \text{number} \rangle \mid \langle \text{number} \rangle \langle \text{multi_digit_number} \rangle$